DEMO 07 · OBSERVABILITY

Anomalies, Summarized — Never Touching Your System

An export-only telemetry analyzer that turns a latency-and-error spike into an anomaly summary with a window and a root-cause candidate — and can do nothing else.

EXPORT-ONLY *Exports summaries — no control*

AUTHOR Rakovskyi Dmytro · VERSION Full-run demo 07 · DATE 2026 · REPOSITORY github.com/rakovpublic/jneopallium

The fastest way to make a monitoring system dangerous is to let it act. A tool that can scale a service, restart a pod, or flip a feature flag in response to what it thinks it sees has become part of the failure surface it was meant to watch. This demo deliberately gives up that power: it reads metrics, traces, and logs, recognises an anomaly, and exports a summary. It cannot write back to anything, by construction.

What it is

Demo 07 models an OpenTelemetry-style anomaly summarizer for a service, `service-checkout`. Its safety ceiling is **EXPORT-ONLY**: it consumes telemetry and emits anomaly records, and there is no result type in its vocabulary that controls or writes back to the monitored system. The network is three layers (sized 4·3·2) — enough to turn raw signals into a feature-level anomaly and a summary.

How it is wired

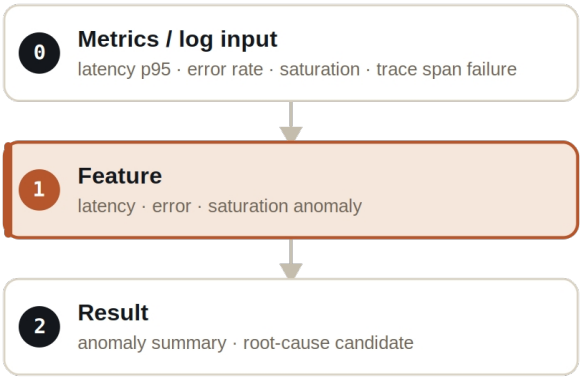

Figure *Three layers. The feature layer detects per-dimension anomalies; the result layer summarizes them with a window.*

Signals are typed: `MetricSignal` carries latency, error rate, and saturation, `TraceSignal` carries span failures, `LogEventSignal` carries log-derived events, and `ObservabilityAnomalySignal` carries the summary. None of these can express a control action.

What it does

- A baseline window produces only normal telemetry export — the analyzer does not invent incidents.
- A spike in latency and error rate across ticks 6 through 12 produces an ANOMALY_SUMMARY that names the window start and end.
- Each anomaly summary carries a root-cause candidate to point an engineer at the likely dimension — never an action.

> **NOTE**
> The summary includes a window start and end, so the anomaly is a bounded interval an engineer can investigate — not a vague, perpetual red light.

The evidence

VERIFICATION EVIDENCE	
Ticks	90 / 90
Spike, ticks 6–12	ANOMALY_SUMMARY with window start/end
Baseline window	NORMAL_TELEMETRY_EXPORT
Writeback / control	**none** — enforced
Output mode	**EXPORT-ONLY**

How to run it

1 Build

Build the worker and the demo model JAR.

```
mvn -q -pl worker -am package
```

2 Run via the framework entry point

Entry loads the model JAR and the demo context through the real local path.

```
java -cp demo-model.jar \
  com.rakovpublic.jneuropallium.worker.application.Entry \
  local file:///path/to/demo-model.jar \
  com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \
  context.json
```

3 Inspect

Read the exported anomaly summaries.

```
ls target/jneopallium-fullrun-demos/demo-07-observability/
```

Why EXPORT-ONLY

Auto-remediation is attractive right up to the first time a monitoring system amplifies an outage by acting on a misread signal. The export-only ceiling keeps this demo strictly an observer: it produces records for your pipeline and dashboards and nothing more. To run it on live telemetry you replace the mock metrics edge with a real OpenTelemetry collector while preserving the typed signal contract and the export-only ceiling — and the guarantee that it never writes back to your services travels with it.